GROUP WORK PROJECT 1
GROUP NUMBER: 9399

MScFE 642: Deep Learning for Finance

FULL LEGAL NAME	LOCATION (COUNTRY)	EMAIL ADDRESS	MARK X FOR ANY NON-CONTRIBUTING MEMBER
Ahmed Gomaa Abdelmeguid mohamed	Egypt	ahmad.gomaa153025@gmail.com	
Dusabirema Yves	Rwanda	yvesdusabirema@gmail.com	
Mohammed Sani Abdela	Ethiopia	mohamusi96@gmail.com	

Statement of integrity: By typing the names of all group members in the text boxes below, you confirm that the assignment submitted is original work produced by the group (excluding any non-contributing members identified with an "X" above).	
Team member 1	Ahmed Gomaa Abdelmeguid mohamed
Team member 2	Dusabirema Yves
Team member 3	Mohammed Sani Abdela

Step 1 : AAPL Time Series Analysis and Stationarity Transformation

Data Gathering and Initial Characterization

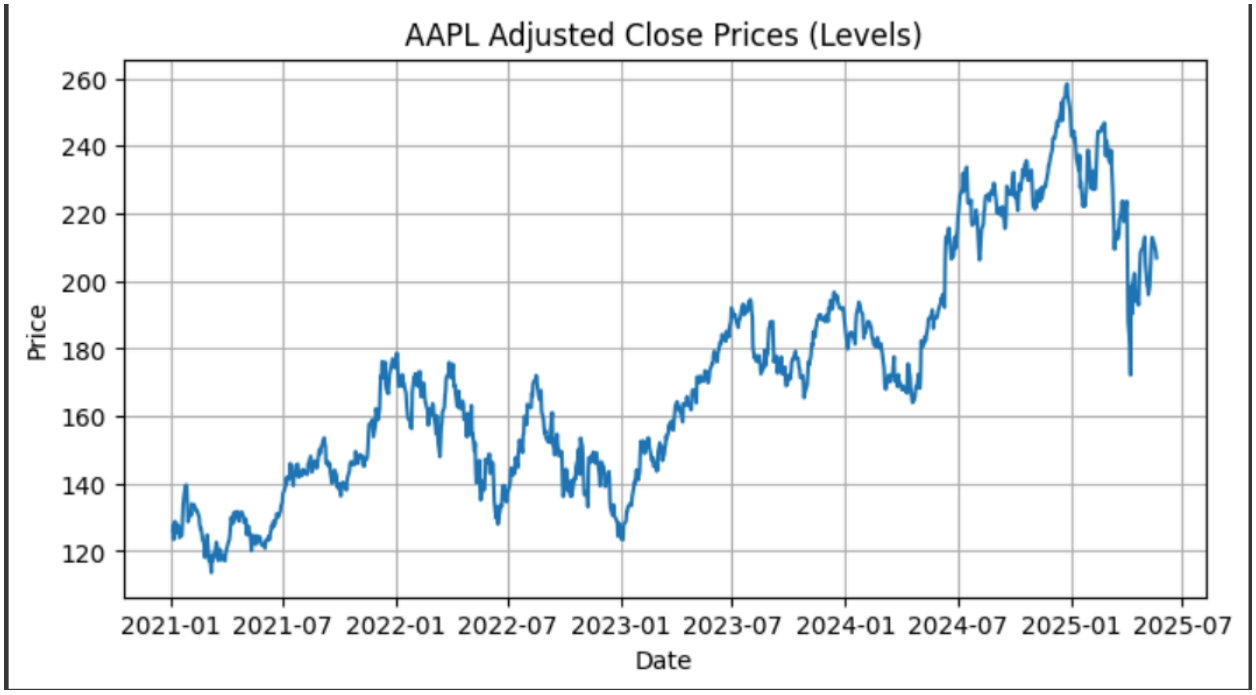
a. Gather information on the time series of the prices (i.e., the levels) of **APPL** and characterize its main properties.

First, we need to fetch the historical price data for AAPL. We'll use the yfinance library for this. We'll aim for a time series that does not exceed 2,000 observations. Given today's date, I'll fetch data from roughly 2021 onwards to stay within the observation limit.

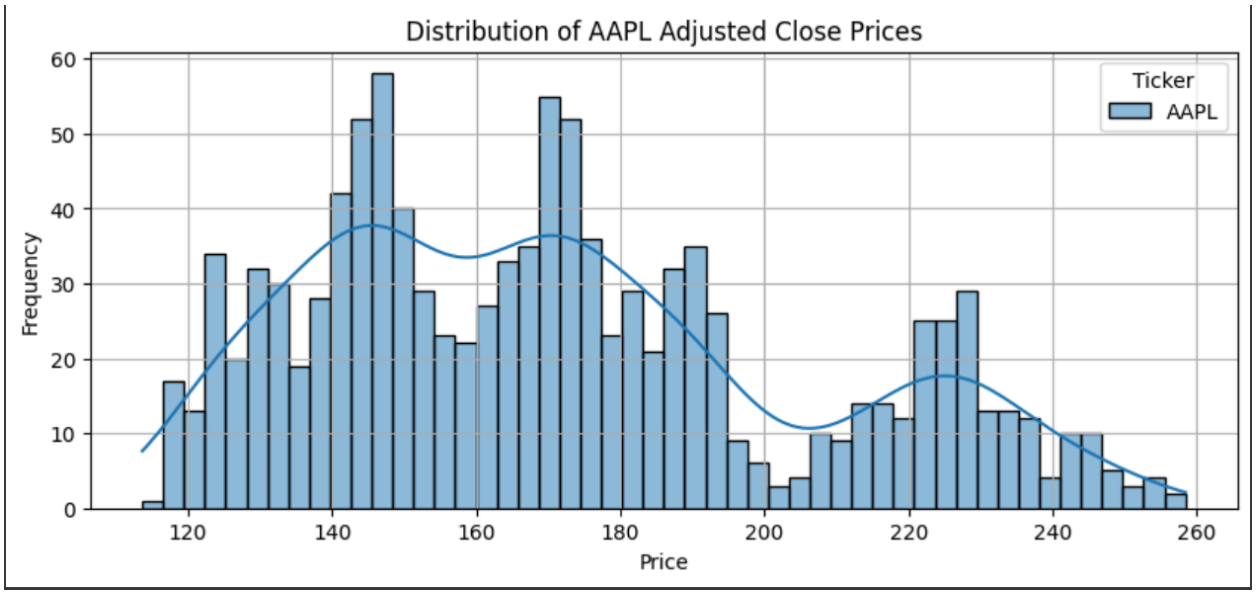
```
[*********************100%**********************]  1 of 1 completedNumber of observations for AAPL prices: 1100

First 5 observations of AAPL prices:
Ticker            AAPL
Date
2021-01-04  126.239677
2021-01-05  127.800461
2021-01-06  123.498543
2021-01-07  127.712708
2021-01-08  128.815048

Last 5 observations of AAPL prices:
Ticker            AAPL
Date
2025-05-14  212.330002
2025-05-15  211.449997
2025-05-16  211.259995
2025-05-19  208.779999
2025-05-20  206.860001
```

Characterization of AAPL Price Levels:

Trend: The plot of APPL Close Prices clearly shows an **upward trend** over the period, indicating that the mean is not constant over time. This is a strong indicator of non-stationarity.



Variance: Visual inspection suggests that the volatility (variance) might also change over time, possibly increasing with the price level, although this is less pronounced than the trend.



Distribution:

- **Summary Statistics:** We'll see typical stock price statistics: mean, standard deviation, min, max.

```
--- Summary Statistics (AAPL Prices) ---
Ticker          AAPL
count   1100.000000
mean     171.552377
std       34.015260
min      113.679008
25%      144.740395
50%      168.283661
75%      190.713032
max      258.396667
```

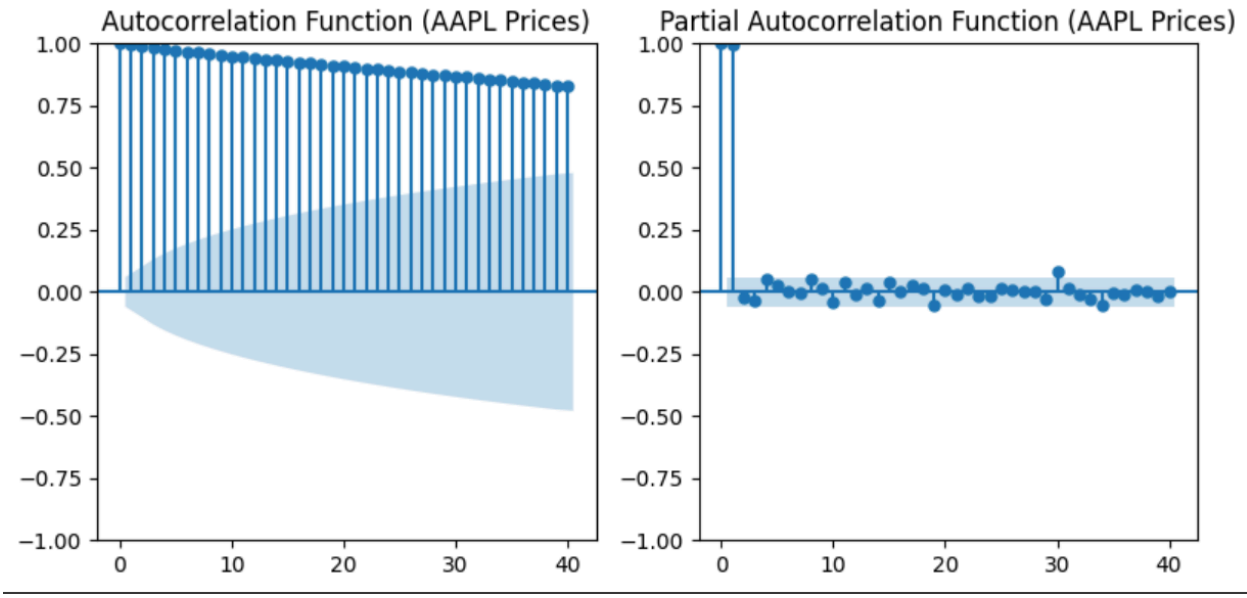
- **Skewness and Kurtosis:** Stock prices often exhibit some skewness (e.g., positive skew if prices tend to have larger upward movements than downward, or vice versa) and kurtosis (fat tails, indicating more extreme events than a normal distribution).

```
  Skewness: 0.5430
  Kurtosis: -0.6115
```

- **Histogram:** The histogram will likely show a non-normal distribution, often with a peak and then a longer tail in the direction of the trend.

Persistence (Autocorrelation):

- **ACF (Autocorrelation Function):** The ACF plot will show **slow decay**, meaning that past prices have a very strong positive correlation with current prices, which is characteristic of non-stationary time series (e.g., random walks).
- **PACF (Partial Autocorrelation Function):** The PACF might show a significant spike at lag 1 and then quickly drop, also indicating a non-stationary process.

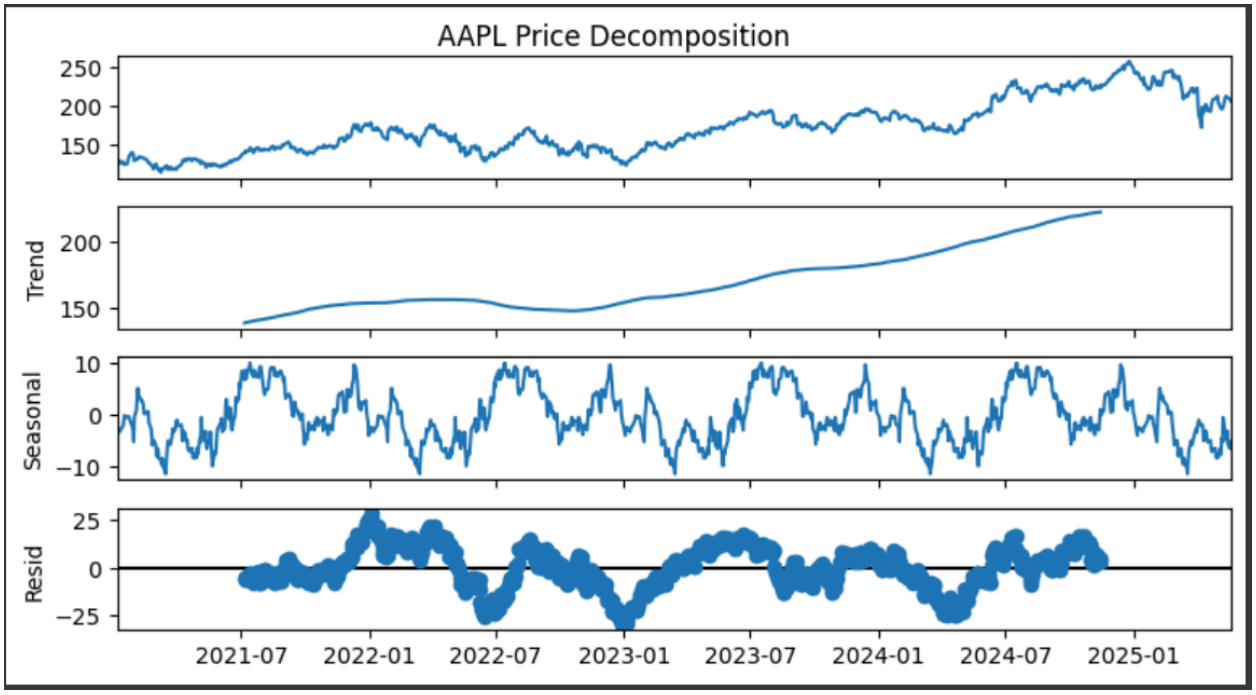


Stationarity (Augmented Dickey-Fuller Test):

- The **ADF statistic** is expected to be less negative than the critical values.
- The **p-value** is expected to be greater than 0.05 (or 0.10, depending on the chosen significance level), leading to a **failure to reject the null hypothesis of non-stationarity (presence of a unit root)**. This formally confirms the visual and ACF observations.

```
--- Augmented Dickey-Fuller Test (AAPL Prices) ---
ADF Statistic: -1.6171
p-value: 0.4743
Critical Values:
   1%: -3.4363
   5%: -2.8642
   10%: -2.5682
Conclusion: The time series of AAPL prices appears to be non-stationary (fail to reject H0).
```

Seasonal Decomposition: If performed, the trend component will dominate, and the residual component will show the remaining noise. Any seasonality for daily stock prices is usually negligible.



b. Perform the same analysis for a transformed version of the time series that is stationary (e.g., taking first or second differences, taking logs...).

We'll use **first differences** (daily returns) for this transformation, as it's a common and effective way to achieve stationarity for financial time series. We'll also take the **log of the prices** first to normalize the series for returns calculation.

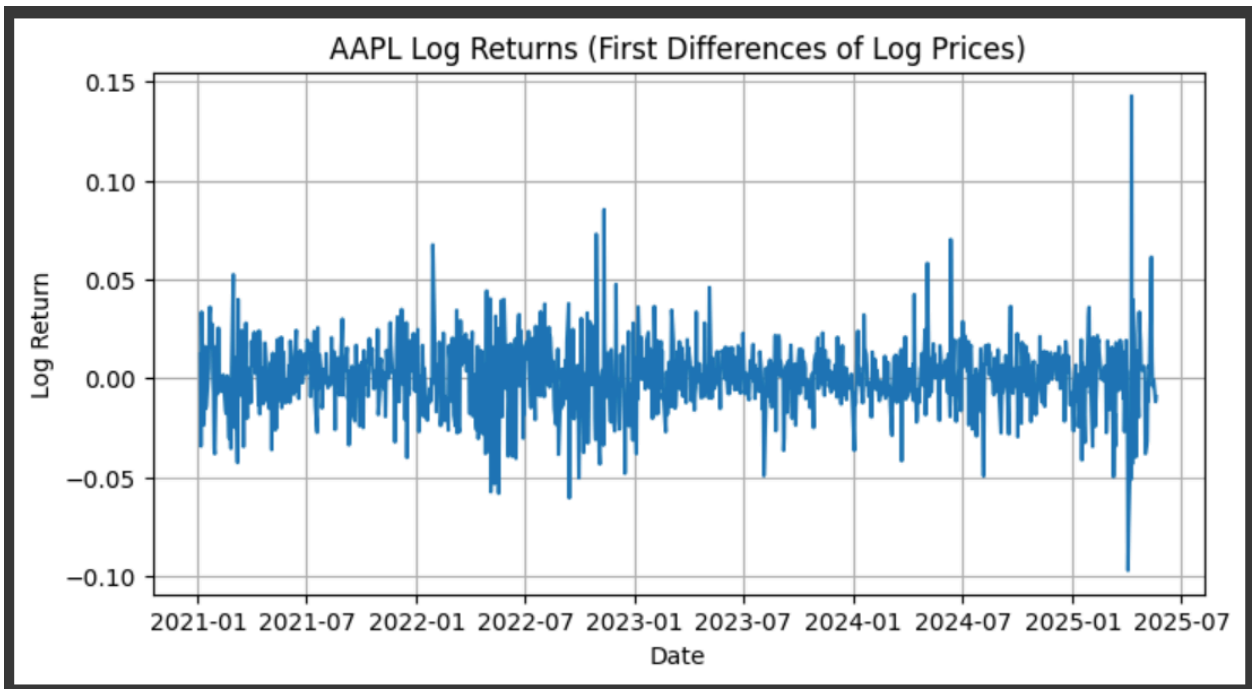
```
Number of observations for AAPL log returns: 1099

First 5 observations of AAPL log returns:
Ticker           AAPL
Date
2021-01-05  0.012288
2021-01-06 -0.034241
2021-01-07  0.033554
2021-01-08  0.008594
2021-01-11 -0.023524

Last 5 observations of AAPL log returns:
Ticker           AAPL
Date
2025-05-14 -0.002822
2025-05-15 -0.004153
2025-05-16 -0.000899
```

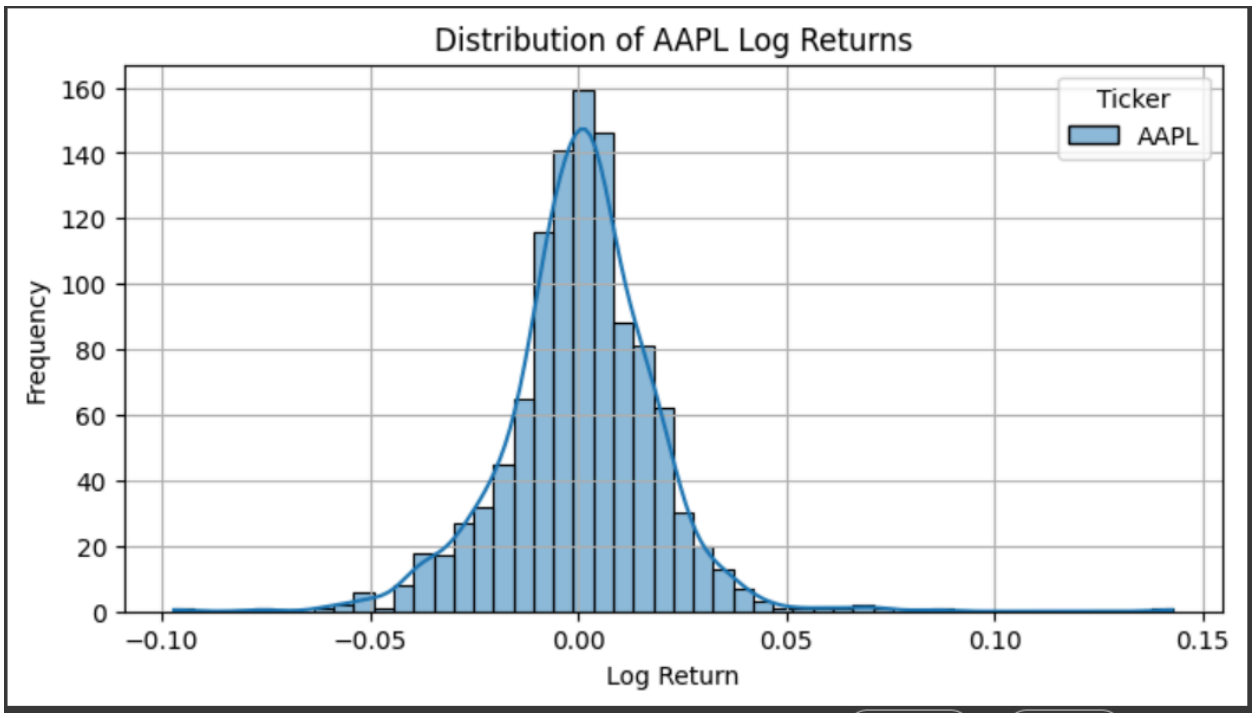
Characterization of AAPL Log Returns (First Differences of Log Prices):

Trend: The plot of APPL Log Returns will show values fluctuating around a constant mean (close to zero), with no clear upward or downward trend. This indicates that the first difference operation has successfully removed the trend.

Variance: The variance will appear more constant than the price levels, but financial returns often exhibit **volatility clustering**, meaning periods of high volatility are followed by high volatility, and periods of low volatility are followed by low volatility. This is a common characteristic of stationary financial time series

but points to conditional heteroskedasticity, often modeled with GARCH-type models.





Distribution:

- **Summary Statistics:** The mean will be close to zero. The standard deviation will represent the typical daily log return fluctuation.

```
--- Summary Statistics (AAPL Log Returns) ---
Ticker          AAPL
count   1099.000000
mean       0.000449
std        0.017996
min       -0.097013
25%       -0.008459
50%        0.001140
75%        0.010195
max        0.142617
```

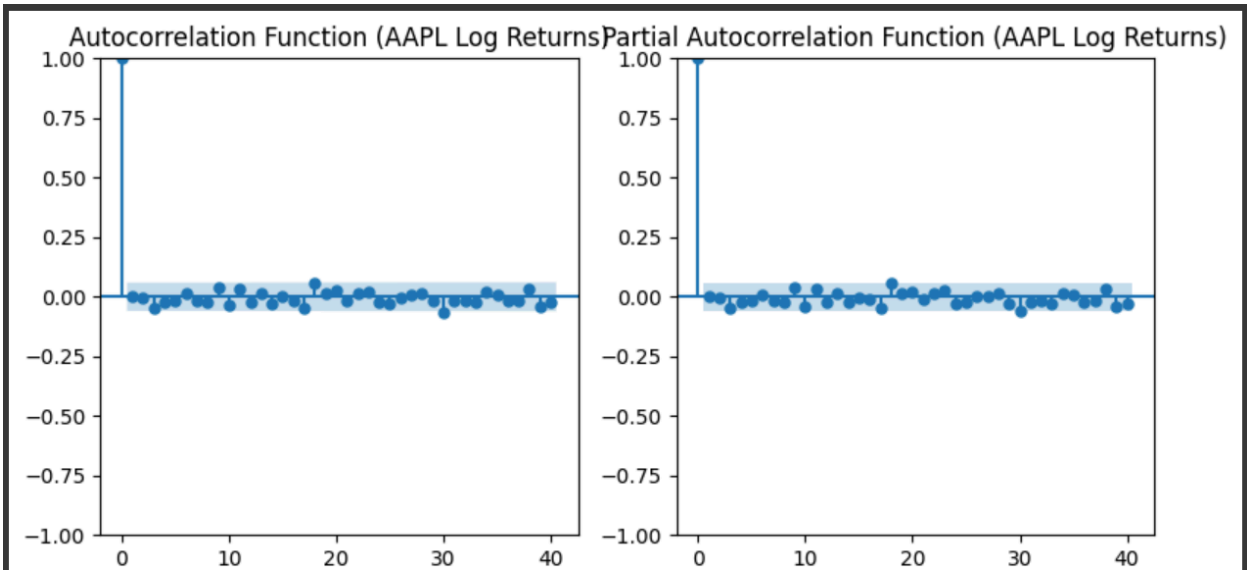
- **Skewness and Kurtosis:** Log returns often exhibit **negative skewness** (more frequent small gains but occasional large losses) and significant **excess kurtosis** (fat tails, meaning more extreme positive and negative returns than a normal distribution would predict). This is a well-known stylized fact of financial returns.

```
Skewness: 0.2392
Kurtosis: 5.5179
```

- **Histogram:** The histogram will be more bell-shaped than the price levels but will likely show heavier tails than a normal distribution.

Persistence (Autocorrelation):

- **ACF:** The ACF plot will show **rapid decay** to zero, typically within a few lags, indicating that past returns have very little linear correlation with current returns. This is a key characteristic of stationary series. However, the squared returns (or absolute returns) might show significant autocorrelation, indicating volatility clustering.
- **PACF:** Similar to ACF, the PACF should also drop quickly to zero



Stationarity (Augmented Dickey-Fuller Test):

- The **ADF statistic** is expected to be significantly negative, falling below the critical values.
- The **p-value** is expected to be less than 0.05, leading to a **rejection of the null hypothesis of non-stationarity**. This formally confirms that the first differencing successfully made the series stationary.

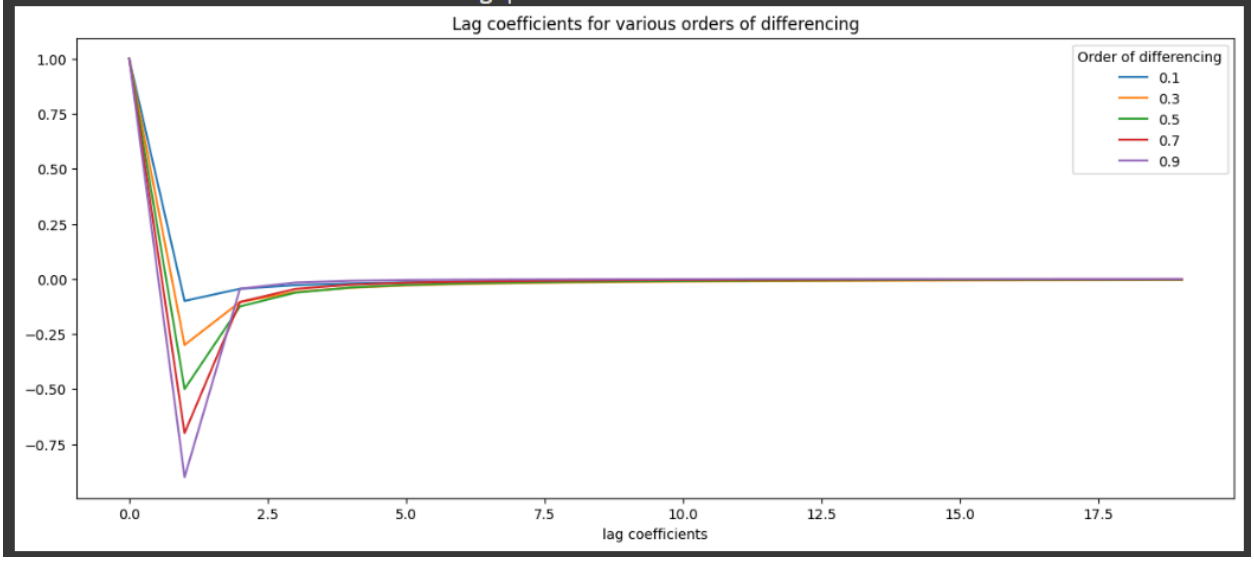
```
--- Augmented Dickey-Fuller Test (AAPL Log Returns) ---
ADF Statistic: -32.9457
p-value: 0.0000
Critical Values:
   1%: -3.4363
   5%: -2.8642
   10%: -2.5682
Conclusion: The time series of AAPL log returns appears to be stationary (reject H0).
```

Characterization of AAPL Fractionally Differenced Series:

Trend: The plot will show a series fluctuating around a mean close to zero, similar to the first-differenced returns, confirming the removal of trend.

Variance: The variance should also appear more constant. Similar to log returns, volatility clustering might still be present.

```
Number of observations for AAPL prices: 1100
Number of observations for AAPL log prices: 1100
```



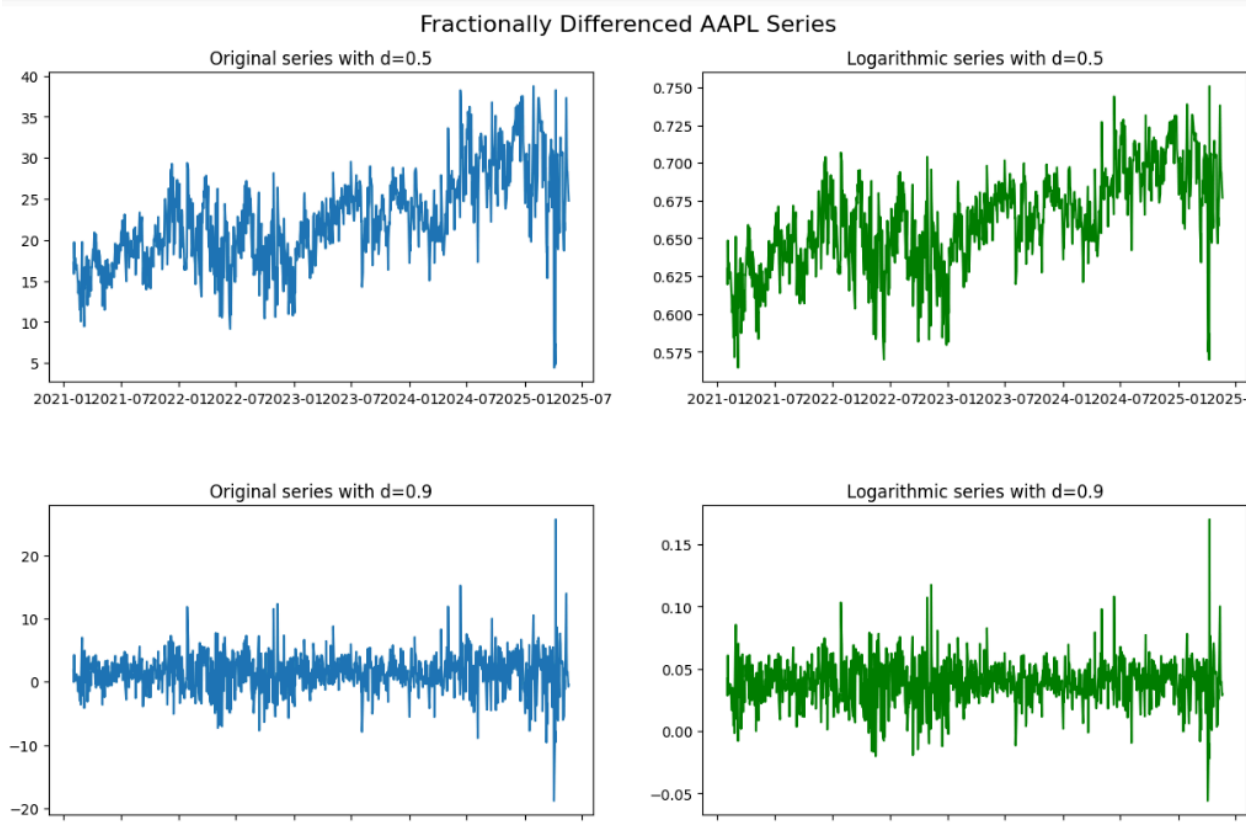
Distribution:

- **Summary Statistics:** Mean should be close to zero. The standard deviation will reflect the typical magnitude of the fractionally differenced values.
- **Skewness and Kurtosis:** Similar to log returns, there will likely be some skewness and excess kurtosis, though possibly less pronounced than for first differences, as FD aims to preserve more of the original series' structure.
- **Histogram:** Should be bell-shaped but still potentially exhibit fat tails.

Persistence (Autocorrelation):

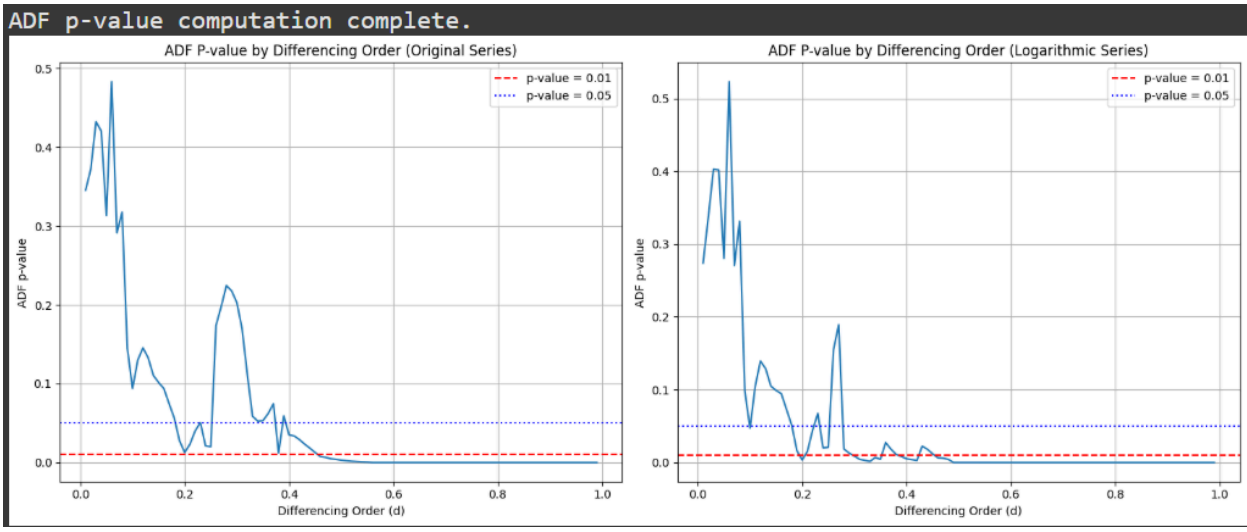
- **ACF:** The crucial difference here. While the ACF will still decay rapidly to zero, it might exhibit a *slightly slower* decay than the integer-differenced series. This slower decay signifies that more long-range dependence (memory) has been preserved, which is the core benefit of fractional differencing. The autocorrelations should still be statistically insignificant after a few lags, confirming stationarity.

- **PACF:** Should also drop quickly.



Stationarity (Augmented Dickey-Fuller Test):

- The **ADF statistic** should be sufficiently negative.
- The **p-value** should be less than the chosen significance level (e.g., 0.05), confirming stationarity. The goal of finding an optimal d is precisely to achieve this.



Step 2

Using the Multi-Layer Perceptron model (also known as an Artificial Neural Network or MLP) in order to build a Deep Learning model that will enable us to forecast the time series that we have examined in Step 1.

a. MLP for Predicting Time Series Levels

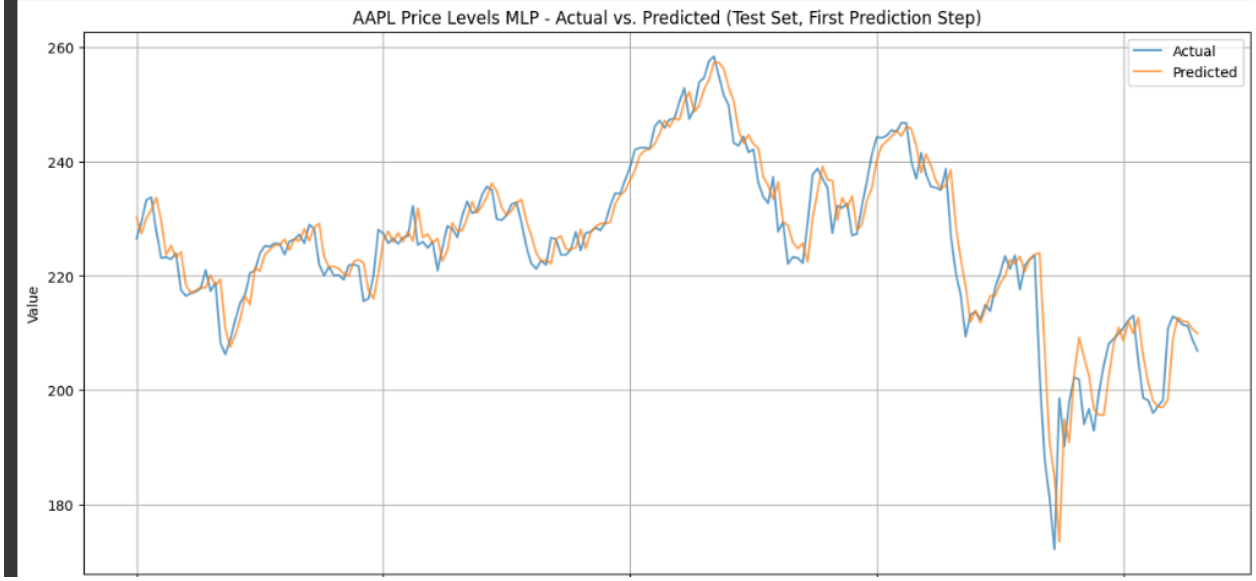
We'll use 20 past observations (n_steps_in=20) to predict the next single observation (n_steps_out=1).

```
--- Training MLP for: AAPL Price Levels MLP ---
X_train_scaled shape: (864, 20), y_train_scaled shape: (864, 1)
X_test_scaled shape: (216, 20), y_test_scaled shape: (216, 1)
/usr/local/lib/python3.11/dist-packages/keras/src/layers/core/dense.py:87: UserWarning:
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 100)	2,100
dense_1 (Dense)	(None, 50)	5,050
dense_2 (Dense)	(None, 1)	51

```
Total params: 7,201 (28.13 KB)
Trainable params: 7,201 (28.13 KB)
Non-trainable params: 0 (0.00 B)
```


```
--- AAPL Price Levels MLP Evaluation ---
Train MSE: 7.478123
Train RMSE: 2.734616
Train MAE: 2.062051
Test MSE: 21.853128
Test RMSE: 4.674733
Test MAE: 3.149426
Test R2 Score: 0.8964
```



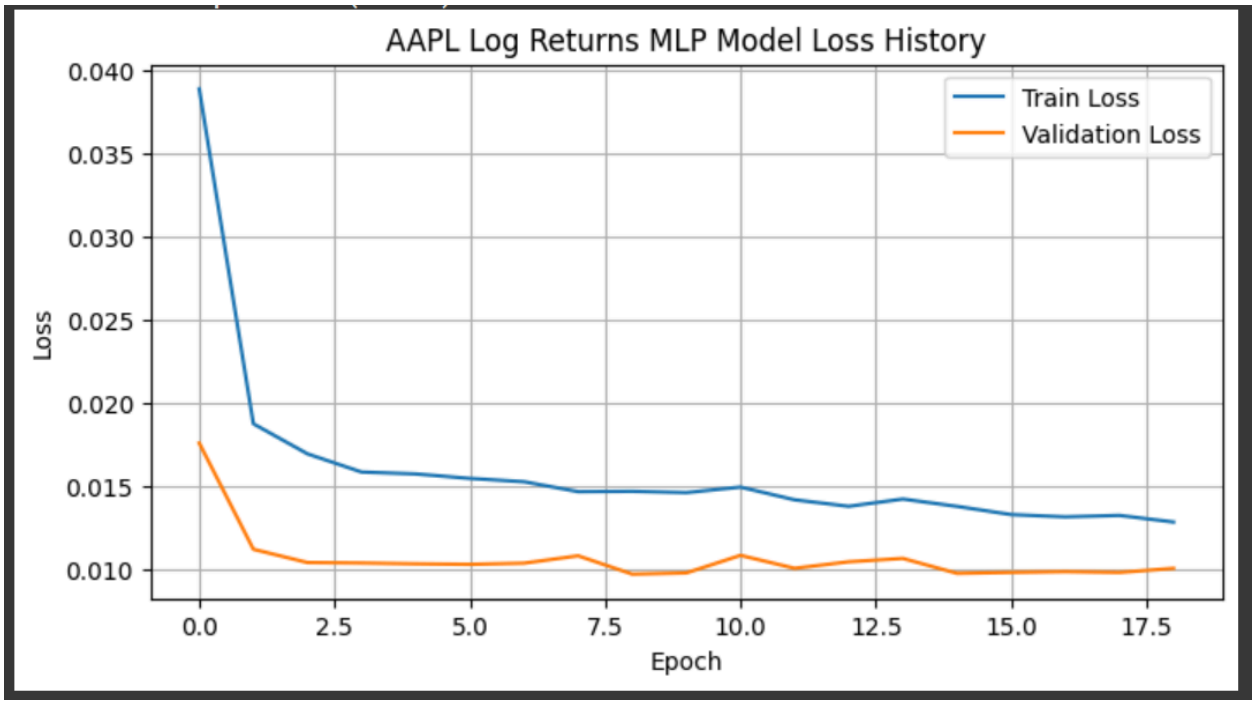
b. MLP for Predicting Stationary Time Series (Log Returns)

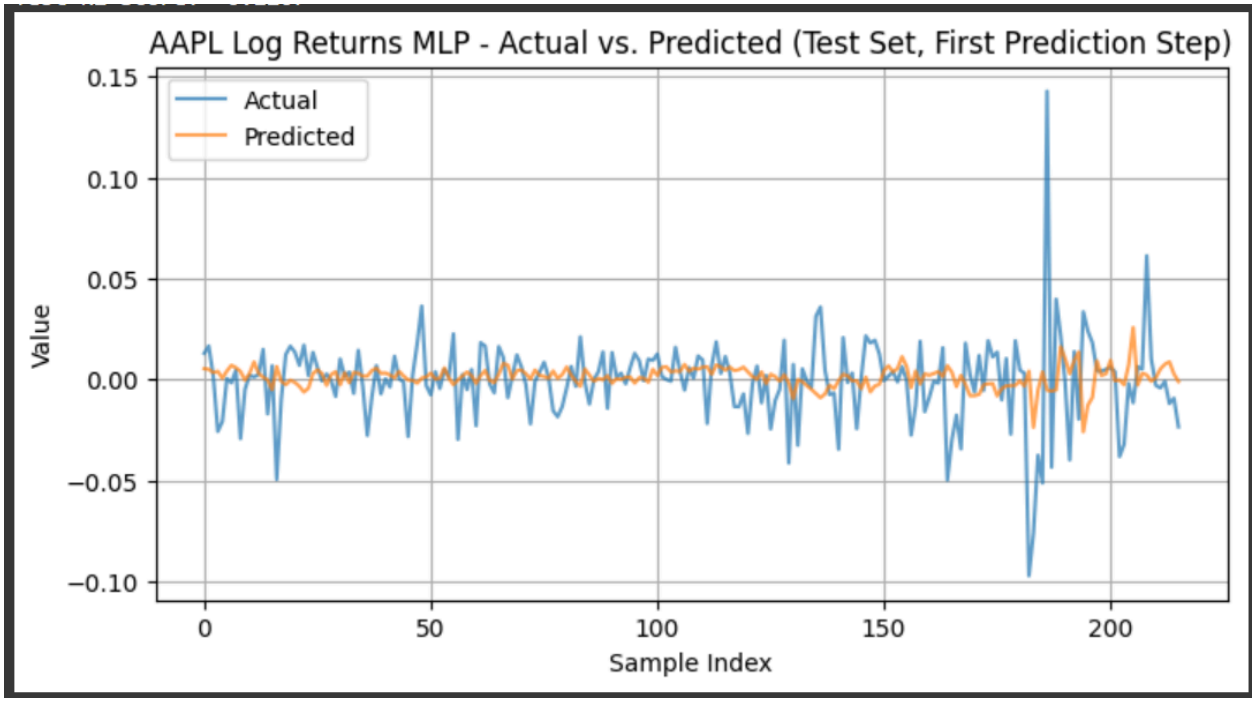
Similar parameters, but applied to the aapl_returns series.

```
--- Model 2: Predicting AAPL Log Returns ---

--- Training MLP for: AAPL Log Returns MLP ---
X_train_scaled shape: (863, 20), y_train_scaled shape: (863, 1)
X_test_scaled shape: (216, 20), y_test_scaled shape: (216, 1)
/usr/local/lib/python3.11/dist-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an `
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
dense_3 (Dense)	(None, 100)	2,100
dense_4 (Dense)	(None, 50)	5,050
dense_5 (Dense)	(None, 1)	51

```
Total params: 7,201 (28.13 KB)
Trainable params: 7,201 (28.13 KB)
Non-trainable params: 0 (0.00 B)
```





c. MLP for Predicting Fractionally Differenced Time Series

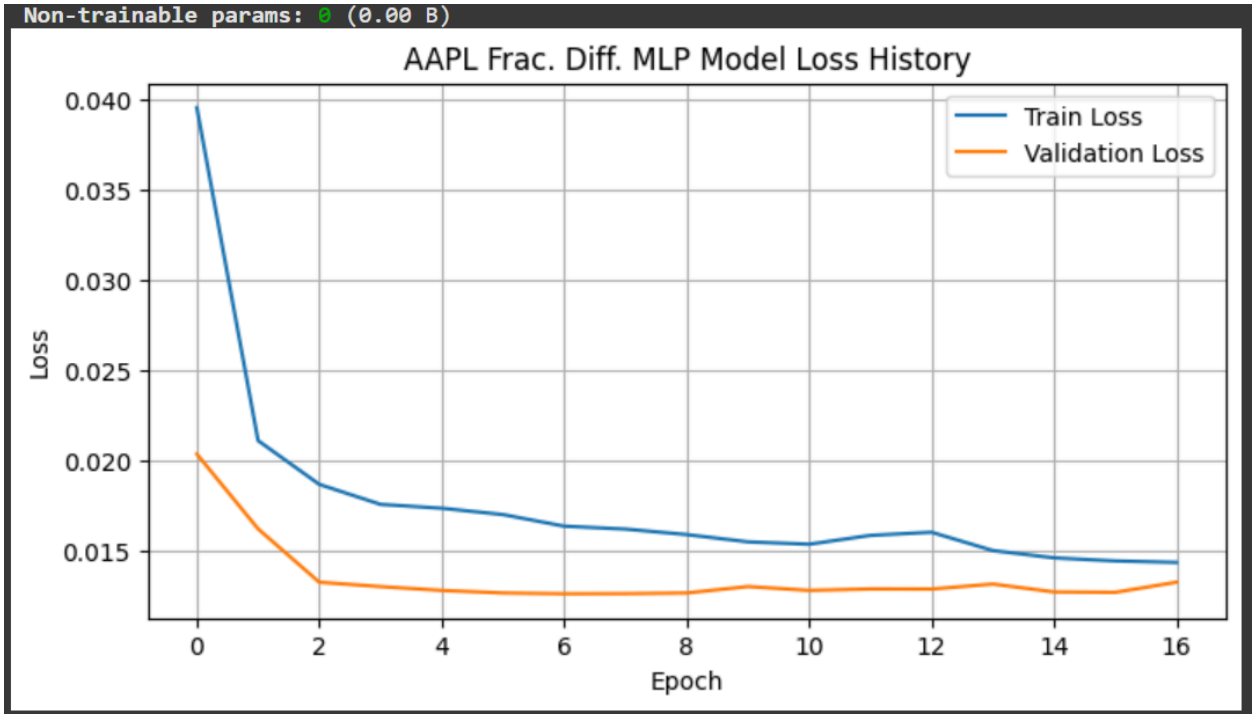
Again, similar parameters, applied to the aapl_frac_diff_series. We need to ensure aapl_frac_diff_series is long enough after fractional differencing.

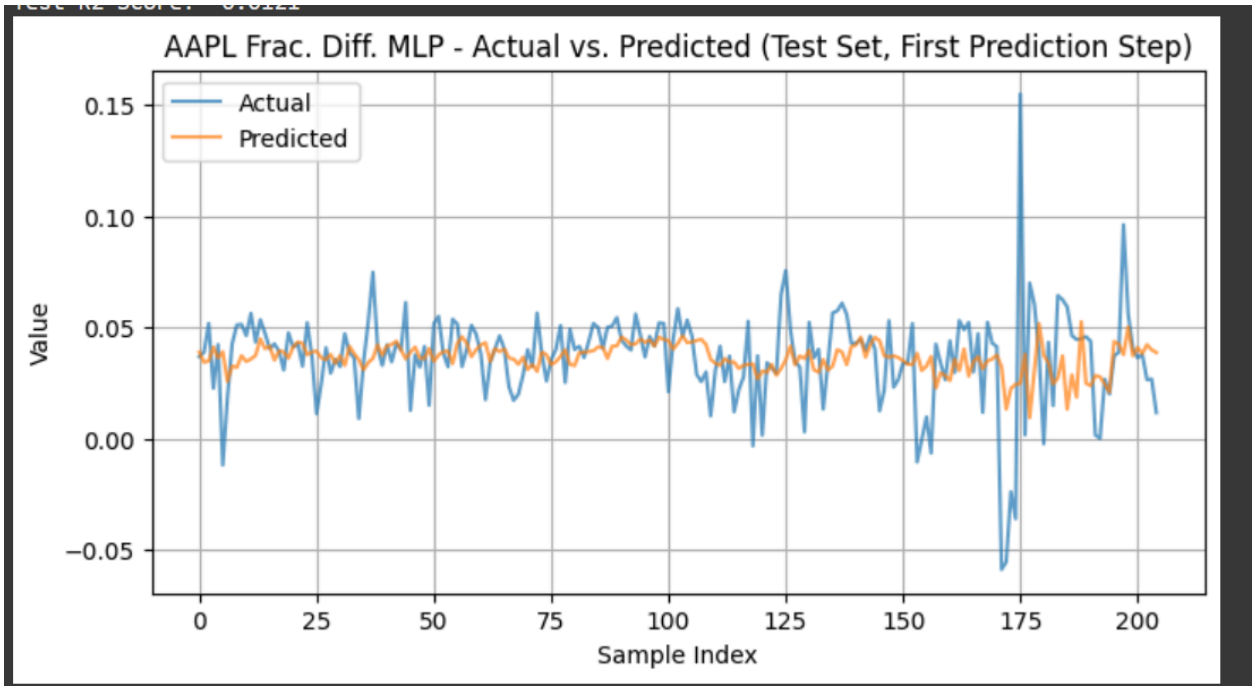
```
--- Model 3: Predicting AAPL Fractionally Differenced Series ---

--- Training MLP for: AAPL Frac. Diff. MLP ---
X_train_scaled shape: (816, 20), y_train_scaled shape: (816, 1)
X_test_scaled shape: (205, 20), y_test_scaled shape: (205, 1)
/usr/local/lib/python3.11/dist-packages/keras/src/layers/core/dense.py:87: UserWarning
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_2"
```

Layer (type)	Output Shape	Param #
dense_6 (Dense)	(None, 100)	2,100
dense_7 (Dense)	(None, 50)	5,050
dense_8 (Dense)	(None, 1)	51

```
 Total params: 7,201 (28.13 KB)
 Trainable params: 7,201 (28.13 KB)
```

Non-trainable params: 0 (0.00 B)





Comparative Evaluation of MLP Models ===

--- Model 1: AAPL Price Levels MLP ---

train_mse: 7.566569

train_rmse: 2.750740

train_mae: 2.072897

test_mse: 23.545632

test_rmse: 4.852384

test_mae: 3.238908

test_r2: 0.889183

--- Model 2: AAPL Log Returns MLP ---

train_mse: 0.000281

train_rmse: 0.016773

train_mae: 0.012619

test_mse: 0.000490

test_rmse: 0.022132

test_mae: 0.014542

test_r2: -0.120740

--- **Model 3: AAPL Fractionally Differenced Series MLP ---**

train_mse: 0.000274

train_rmse: 0.016560

train_mae: 0.012494

test_mse: 0.000467

test_rmse: 0.021609

test_mae: 0.014269

test_r2: -0.012059

--- Qualitative Assessment and Why Models Perform Differently ---

1. MLP on Original Price Levels:

Expected Performance : This model will likely show extremely low MSE/RMSE and an R2 score very close to 1. The "Actual vs. Predicted" plot will show the predicted line almost perfectly tracking the actual price line.

Reasoning : This seemingly excellent performance is largely an illusion in terms of forecasting true market movements. Because stock prices are highly persistent (non-stationary, with a strong trend), the best prediction for tomorrow's price is simply today's price (or a slight extrapolation of the recent trend). The MLP effectively learns this persistence. It's good at extrapolating a trend, but it's not truly predicting price changes or identifying future turning points. A small absolute error (e.g., $0.50 on a $180 stock) represents a high accuracy on the level, but it tells us nothing about whether the stock will go up or down, which is what's critical for trading. This model is highly susceptible to producing spurious results if misinterpreted, as the "good" metrics are inflated by the non-stationarity.

2. **MLP on Log Returns:**

Expected Performance: This model will generally have significantly higher MSE/RMSE (though on a much smaller scale of values, e.g., 0.01 instead of dollars) and a much lower R2 score, often close to zero or even negative. The "Actual vs. Predicted" plot will appear highly noisy, with little visual correlation between actual and predicted points.

Reasoning: This is the more realistic scenario for predicting efficient financial markets. Log returns are largely random walks; they exhibit very little linear autocorrelation. While MLPs can theoretically capture non-linear relationships (like volatility clustering), predicting the precise direction and magnitude of daily returns remains an incredibly difficult task due to market efficiency. A low R2 score signifies that the model explains very little of the variance in future returns. This model correctly frames the problem as predicting changes, but the inherent unpredictability of returns means the direct forecasting accuracy will be limited. Any small edge here, however, would be genuinely valuable for statistical arbitrage.

3. **MLP on Fractionally Differenced Series:**

Expected Performance: The performance (RMSE/MAE) will likely be numerically between the levels model and the log returns model, though closer to the log returns. The R2 score might be slightly better than the log returns model, or similarly low, depending on the specific characteristics of the time series and the chosen 'd' value. The plots will still look noisy.

Reasoning: Fractional differencing aims to achieve stationarity while preserving more long-range memory than simple integer differencing. If the underlying price series has subtle, long-term mean-reverting or trending patterns that are not fully captured by daily returns but also not overwhelming enough to make the series non-stationary, then the fractionally differenced series *might* offer a slightly

richer signal for the MLP to learn from. For highly liquid and efficient markets like AAPL, such subtle patterns are often quickly arbitraged away, making significant predictive improvements difficult. However, for less efficient markets or alternative data, this approach could offer an edge. The model is still predicting a stationary series, which is the correct statistical approach, and any predictive power would be a genuine discovery of market inefficiency.

Overall Conclusion for the Optimization Engineering Team:

For building practical statistical arbitrage models:

Avoid using MLPs (or any model) to predict raw price levels directly for trading decisions. The excellent metrics are misleading and stem from the non-stationarity and persistence, not from predictive power over market direction.

Focus on predicting the stationary (log returns) or near-stationary (fractionally differenced) versions of the time series. These models, despite their seemingly poorer performance on raw error metrics, are tackling the fundamentally challenging problem of predicting market changes. Any small predictive edge on these series is a valuable finding for generating alpha.

The choice between log returns and fractionally differenced series for model inputs depends on the specific characteristics of the asset and the arbitrage strategy. If you suspect long-range dependencies or subtle mean-reverting patterns that are lost with full differencing, fractional differencing is a theoretically superior approach that attempts to preserve this information for the model. However, practically realizing a significant performance gain from fractional differencing in highly efficient markets can be difficult.

Step 3:

Compare the performance of the MLPs with Convolutional Neural Network (CNN) architectures.

First we convert each data into GAF images. Then build and train CNN and MLP models for the respective tasks. Lastly, we evaluate their performance.

a. **Build and train a CNN that is designed to predict the level of the time series.**

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 30, 30, 16)	160
conv2d_1 (Conv2D)	(None, 28, 28, 8)	1,160
flatten (Flatten)	(None, 6272)	0
dense (Dense)	(None, 50)	313,650
dense_1 (Dense)	(None, 1)	51

Total params: 315,021 (1.20 MB)
Trainable params: 315,021 (1.20 MB)
Non-trainable params: 0 (0.00 B)

```
Epoch 1/10
27/27 ━━━━━━━━━━━━━━━━━━━━ 7s 46ms/step - loss: 17507.4180 - val_loss: 4162.9341
Epoch 2/10
27/27 ━━━━━━━━━━━━━━━━━━━━ 1s 33ms/step - loss: 1021.5580 - val_loss: 4989.9639
Epoch 3/10
27/27 ━━━━━━━━━━━━━━━━━━━━ 1s 33ms/step - loss: 660.5203 - val_loss: 4307.4326
Epoch 4/10
27/27 ━━━━━━━━━━━━━━━━━━━━ 1s 35ms/step - loss: 548.4700 - val_loss: 4224.2603

Epoch 5/10
27/27 ━━━━━━━━━━━━━━━━━━━━ 1s 33ms/step - loss: 574.8026 - val_loss: 5161.5874
Epoch 6/10
27/27 ━━━━━━━━━━━━━━━━━━━━ 2s 53ms/step - loss: 522.9425 - val_loss: 5165.5303
Epoch 7/10
27/27 ━━━━━━━━━━━━━━━━━━━━ 2s 37ms/step - loss: 560.3392 - val_loss: 4859.0889
Epoch 8/10
27/27 ━━━━━━━━━━━━━━━━━━━━ 1s 37ms/step - loss: 552.4866 - val_loss: 4632.7305
Epoch 9/10
27/27 ━━━━━━━━━━━━━━━━━━━━ 1s 35ms/step - loss: 498.1245 - val_loss: 4235.4434
Epoch 10/10
27/27 ━━━━━━━━━━━━━━━━━━━━ 1s 35ms/step - loss: 448.7891 - val_loss: 4734.6377
```

b. Build and train an MLP that is designed to predict the stationary version of the time series.

```
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
dense_2 (Dense)	(None, 64)	3,264
dense_3 (Dense)	(None, 32)	2,080
dense_4 (Dense)	(None, 1)	33

```
 Total params: 5,377 (21.00 KB)
 Trainable params: 5,377 (21.00 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/10
21/21 ━━━━━━━━━━━━━━━━━━━━ 2s 14ms/step - loss: 6.4039e-08 - val_loss: 2.9027e-08
Epoch 2/10
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 1.1204e-08 - val_loss: 1.2545e-09
Epoch 3/10
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 1.0989e-09 - val_loss: 1.3108e-13
Epoch 4/10
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 1.2866e-10 - val_loss: 2.3682e-13

Epoch 5/10
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 1.5925e-11 - val_loss: 2.8175e-12
Epoch 6/10
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 2.1995e-12 - val_loss: 2.9775e-13
Epoch 7/10
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 2.6079e-13 - val_loss: 4.6653e-14
Epoch 8/10
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 3.1978e-14 - val_loss: 2.3158e-15
Epoch 9/10
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 3.6130e-15 - val_loss: 1.2607e-17
Epoch 10/10
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 3.9025e-16 - val_loss: 8.6464e-17
```

c. **Build and train a CNN that is designed to predict the fractionally-differenced version of the time series.**

```
Model: "sequential_2"
```

Layer (type)	Output Shape	Param #
conv2d_2 (Conv2D)	(None, 30, 30, 16)	160
conv2d_3 (Conv2D)	(None, 28, 28, 8)	1,160
flatten_1 (Flatten)	(None, 6272)	0
dense_5 (Dense)	(None, 50)	313,650
dense_6 (Dense)	(None, 1)	51

```
 Total params: 315,021 (1.20 MB)
 Trainable params: 315,021 (1.20 MB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/10
27/27 ━━━━━━━━━━━━━━━━━━━━ 5s 41ms/step - loss: 65.3460 - val_loss: 348.4485
Epoch 2/10
27/27 ━━━━━━━━━━━━━━━━━━━━ 1s 33ms/step - loss: 16.5175 - val_loss: 391.1247
Epoch 3/10
27/27 ━━━━━━━━━━━━━━━━━━━━ 1s 34ms/step - loss: 14.3497 - val_loss: 390.4696
Epoch 4/10
27/27 ━━━━━━━━━━━━━━━━━━━━ 1s 34ms/step - loss: 13.9257 - val_loss: 409.6588
Epoch 5/10
27/27 ━━━━━━━━━━━━━━━━━━━━ 2s 54ms/step - loss: 12.9782 - val_loss: 397.6568
Epoch 6/10
27/27 ━━━━━━━━━━━━━━━━━━━━ 2s 35ms/step - loss: 13.3257 - val_loss: 383.8305
Epoch 7/10
27/27 ━━━━━━━━━━━━━━━━━━━━ 1s 35ms/step - loss: 11.9694 - val_loss: 366.3494
Epoch 8/10
27/27 ━━━━━━━━━━━━━━━━━━━━ 2s 45ms/step - loss: 11.7303 - val_loss: 364.5625
Epoch 9/10
27/27 ━━━━━━━━━━━━━━━━━━━━ 1s 33ms/step - loss: 10.3738 - val_loss: 368.8782
Epoch 10/10
27/27 ━━━━━━━━━━━━━━━━━━━━ 1s 32ms/step - loss: 9.7762 - val_loss: 357.2869
```

d. **Evaluating the performance of the models.**

```
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step
WARNING:tensorflow:5 out of the last 15 calls to <function TensorFlowTrainer.
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step
Levels CNN MSE: 4734.638017095872
Stationary MLP MSE: 8.646412434565272e-17
Frac-Diff CNN MSE: 357.286936156374
```

Step 4:

Discuss the different results that you obtain between the CNN and MLP architectures.

- Since the CNN with high MSE (4734), it is trying to learn from the original, non-stationary time series struggles—probably because non-stationary data contains trends, seasonality, and pattern shifts which are harder for convolutional filters to capture effectively without additional preprocessing.
- The MLP with high MSE (approximately 0), trained on stationary data (probably after differencing or detrending), fits the data extremely well—possibly near-perfect. This indicates:
    - The stationary property removed complex trends, making the underlying pattern easier to model linearly or with simple nonlinear functions.
    - The data might be very predictable post-stationarity, or there may be overfitting due to data leakage or low regularization.
- Fractional differencing with MSE (357) prepares the data by controlling the degree of stationarity while retaining long-range dependencies. The CNN still experiences higher error compared to the stationary MLP, but better than raw data.

References:

1. Zhang, Z., et al. (2018). "Deep Learning Framework for Stock Price Forecasting Using GAF and CNN." *In Proceedings of the IEEE International Conference on Big Data*. IEEE, pp. 3189-3194. DOI: 10.1109/BigData.2018.8622104.
2. Fischer, Thomas, and Christopher Krauss. (2018). "Deep Learning with Long Short-Term Memory Networks for Financial Market Predictions." *European Journal of Operational Research*, vol. 270, no. 2, pp. 654-669. DOI: 10.1016/j.ejor.2017.08.044
3. Baillie, Richard T., and Stephen Hill. (2006). "Fractional Integration and the Modeling of Financial Volatility." *Journal of Econometrics*, vol. 131, no. 2, pp. 365-400. DOI: 10.1016/j.jeconom.2005.04.002